4. The tester uses the results of these tests as the starting point for manual and exploratory tests.
5. The automated tests act as low-level technical documentation, and provide up-to-date examples of how the system works. Sarah can review the test reports to see what features have been delivered, and whether they work the way she expected.

Compared to Chris's scenario, Sarah's team makes heavy use of conversations and examples to reduce the amount of information lost in translation. Every stage beyond step 2 starts with the specifications written in Gherkin, which are based on concrete examples provided by Sarah. In this way, a great deal of the ambiguity in translating the client's initial requirements into code, reports, and documentation is removed.

We'll discuss all of these points in detail throughout the rest of the book. You'll learn ways to help ensure that your code is of high quality, solid, well tested, and well documented. You'll learn how to write more effective unit tests and more meaningful automated acceptance criteria. You'll also learn how to ensure that the features you deliver solve the right problems and provide real benefit to the users and the business.

1.2 What problems are you trying to solve?

Software projects fail for many reasons, but the most significant causes fall into two broad categories:

- Not building the software right
- Not building the right software

Figure 1.3 illustrates this in the form of a graph. The vertical axis represents *what* you're building, and the horizontal axis represents *how* you build it. If you perform poorly on the *how* axis, not writing well-crafted and well-designed software, you'll end up with a buggy, unreliable product that's hard to change and maintain. If you don't do well on the *what* axis, failing to understand what features the business really needs, you'll end up with a product that nobody needs.

1.2.1 Building the software right

Many projects suffer or fail because of software quality issues. Although internal software quality is mostly invisible to nontechnical stakeholders, the consequences of poor-quality software can be painfully visible. In my experience, applications that are poorly designed, badly written, or lack well-written, automated tests tend to be buggy, hard to maintain, hard to change, and hard to scale.

I've seen too many applications where simple change requests and new features take too long to deliver. Developers spend more and more time fixing bugs rather than working on new features, which makes it harder to deliver new features quickly. It takes longer for new developers to get up to speed and become productive, simply because the code is hard to understand. It also becomes harder and harder to add new features without breaking existing code. The existing technical documentation (if there is any) is inevitably out of date, and teams find themselves incapable of